Big Data to Boost the work of Software Development

Gebeyehu B. (Phd), Aemro Enyew, Belaynesh Mussie, Habtamu Adugna

Abstract
In the contemporary landscape of software engineering, the optimization of workflow efficiencies and informed decision-making processes stand as pivotal objectives for ensuring the success of projects and the delivery of high-quality software products. This research investigates the application of big data analytics in enhancing software development practices, specifically through the analysis of Agile Scrum Velocity and system log datasets. Utilizing quantitative methods, we examine the impact of Scrum velocity data on team productivity, workflow efficiency, and project planning. Additionally, we analyze log data from distributed systems to identify anomalies, understand user behavior, and optimize service performance. Our findings reveal significant insights into improving software development outcomes, emphasizing the importance of data-driven decision-making and proactive system maintenance.

Keywords - Big Data, Big Data Analytics, software development, Agile Scrum, Scrum Velocity, Sprint Analysis, Log Data Analysis, Anomaly Detection, User Behavior Analysis, Predictive Maintenance

1. Introduction

The integration of big data and analytics has emerged as a transformative force across various industries, offering exceptional opportunities to optimize processes, enhance decision-making, and drive innovation. One domain where this convergence holds a huge promise is software development. As organizations increasingly rely on software to drive their operations and deliver value to customers, the need for efficient, reliable, and scalable development practices becomes ever more critical. The agile software development life cycle is based on the concept of incremental model and agile development does not promote best practices; rather, agile is about adaptive planning and evolutionary development. The availability of data and the current advancements in software analytics enable software project management to gain insights and make decisions driven by the analytical results [8].

One way to improve software development is by analyzing the data it generates. A prime example of this data is the velocity dataset from Scrum methodologies. Scrum is an agile, lightweight framework that provides steps to manage and control the software and product development process. It is highlighted as the most commonly used method for software product development among various agile methods like Crystal Methodologies, Dynamic Software Development Method (DSDM), and Feature Driven Development (FDD) [1].

A Scrum velocity data set is a collection of historical information related to the amount of work completed by a Scrum development team during each Sprint. Scrum velocity refers to the amount of work completed per sprint. It is a key metric used in Scrum to measure the team's productivity and predict future outcomes. The historical sprint velocity data is crucial for creating reliable project schedules and setting stakeholders' expectations [2]. In summary, analyzing Scrum velocity is crucial for predicting future outcomes, identifying problems, improving project planning, and enhancing project assessment within the Scrum framework [2].

Furthermore, software systems generate a wealth of operational data during their runtime, which is commonly captured through logging practices. As a technique to collect the runtime information of system behavior, logging practice is more and more important in modern software development and maintenance [6].

Nowadays modern software systems are getting larger and more distributed, making it more challenging to detect anomalies and locate the corresponding root causes particularly in intricate software systems. Information such as runtime behavior of software systems is thus believed critical by many researchers and practitioners in detecting and addressing anomalies. To ensure the quality and the health of the deployed systems, software logging plays a central role [5]. The extensive analysis of log data in distributed systems is fundamental for understanding system behavior, anomaly detection, user interaction patterns, and predictive maintenance needs. our research intends to

contribute valuable insights into the analytical findings by synthesizing the integral data-driven approaches and the empirical study on logging practices

Our research aims to boost the work of software engineering by synthesizing data-driven approaches and empirical studies on development process data and log files. By leveraging Scrum velocity datasets and log data analysis, we seek to uncover insights that can inform decision-making, enhance workflow efficiencies, and improve overall software development outcomes. Through a comprehensive exploration of these data-driven methodologies, we endeavor to contribute valuable insights that can drive innovation and advance best practices within the field of software engineering.

2. Research questions

The objective of this research is to answer the following questions

a) To what extent does analyzing Agile Scrum Velocity Dataset impact the identification of workflow inefficiencies and improvement opportunities within software development teams?
b) How does the utilization of Agile Scrum Velocity Dataset influence decision-making and planning processes within software development teams?
c) How Log data analytics are used for detecting anomalies in system logs, and how can these insights be utilized for proactive and predictive system maintenance in software development?
d) What insights can be gained by software developers from user behavior analysis and monitoring from big data?
e) How can big data analytics be applied to analyze and optimize service response times, considering variations and fluctuations within each service?

3. Methodology

Data Collection Method: In this study, we leveraged two data sets from online repositories

a. The Agile Scrum Velocity Dataset hosted on GitHub [3]
b. Log data from a distributed system, obtained from a Kaggle dataset

Research Method: The Research Design is Observational **and** the research Approach is Quantitative**.** The research is observational as it involves observing and analyzing pre-existing data without manipulating the study environment or conditions. This method allows for the identification and analysis of patterns, trends, insights, and anomalies within scum and log data collected from online repositories. The research approach is quantitative because it involves the systematic empirical investigation of observable phenomena via statistical, mathematical, or computational techniques. Specifically, the dataset comprises numerical data which are analyzed using various quantitative methods.

Materials/Tools

Python/ML libraries: pandas, numpy, seaborn, matplotlib, scikit-learn, Jupyter Notebook

4. Data Analysis

The data analysis phase stands as the main focal-point of our research where we get the answer to our research questions. To answer our research question, we have carried out analysis on two data sets as follows.

4.1.Agile Scrum Velocity Dataset Analysis

A. Data Set

The Agile Scrum Velocity dataset comprises three CSV files containing different data for each sprint in the Spring XD project. Furthermore, the dataset includes three text files defining the terminology employed within the CSV files.

Among the data sets, spring 67.csv dataset provides comprehensive information about 66 individual sprints within a software development project. Each row represents a specific sprint, and the dataset contains various attributes describing the characteristics and performance of each sprint.

B. Analysis

i. Sprint Velocity Trend Analysis

Sprint velocity is typically calculated based on the number of completed issues within a sprint and the duration of the sprint. Fig 1 shows sprint velocity (completed issues count) over time (sprint start date or complete date) to visualize the trend in team productivity and patterns in sprint velocity over multiple sprints. The mean and variance are 2.37 and 1.97 respectively. Variance of 1.97 is relatively high compared to the mean (2.37). This suggests that there's a spread in the velocities across different sprints.

The Sprint Velocity Trend Analysis across 66 sprints reveals fluctuations in sprint velocity over time, with a noticeable decrease observed after sprint 50. This trend suggests potential challenges or bottlenecks affecting team productivity, which needs further investigation. The decrease in sprint velocity may indicate issues such as scope creep, inadequate resource allocation, or process inefficiencies that require attention to maintain project momentum and meet delivery deadlines.

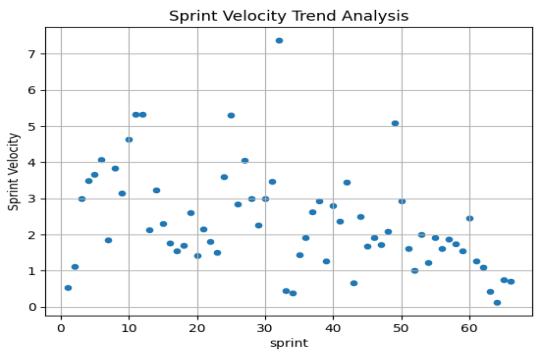

Figure 1 Sprint Velocity Trend Analysis

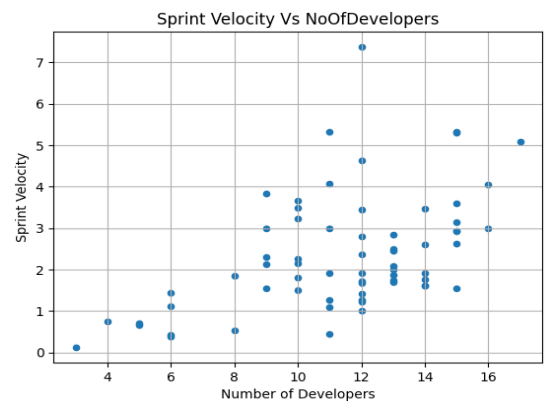

Figure 2 Sprint Velocity vs Number of Developer

ii. Team Composition Analysis

The focus of this analysis is to find out the relationship between team size (number of developers) and sprint velocity. Fig 2 shows sprint velocity against the number of developers to understand how team composition affects productivity.

When exploring the relationship between sprint velocity and the number of developers involved, it becomes apparent that the impact of team composition on productivity is not straightforward. While there is a general trend of higher sprint velocities associated with larger teams (8-18 developers), there is considerable variability in sprint velocity within each group. For instance, with 12 developers, the range of sprint velocity spans from 1 to 8, indicating that other factors beyond team size may influence productivity. This observation suggests that a more subtle understanding of team dynamics, such as skill mix, communication effectiveness, and task allocation, may be necessary to optimize team performance.

iii. On-Time Sprint Analysis

On-Time sprints are those sprints where the sprintCompleteDate is on or before the planned sprintEndDate. Sprint Slippage is the amount of time by which a sprint's actual completion date (CompleteDate) exceeds its planned end date (EndDate). Fig 3 shows sprint slippage trend, Fig 4 shows the relationship between sprint slippage and number of developers.

The analysis of sprint slippage reveals interesting insights into the timeliness of sprint completion. Initially, there is evidence of sprint slippage, but it gradually diminishes over time as the team gains experience and refines its processes. However, the relationship between sprint slippage and the number of developers follows a non-linear pattern, characterized by an initial increase followed by a decrease, resembling a dome shape. This finding suggested the appropriate number of developers to decrease sprint slippage.

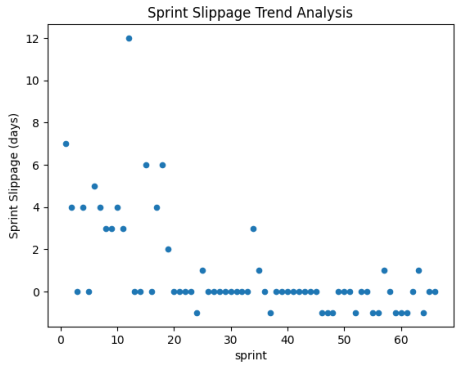

Figure 3 Sprint Slippage Trend Analysis

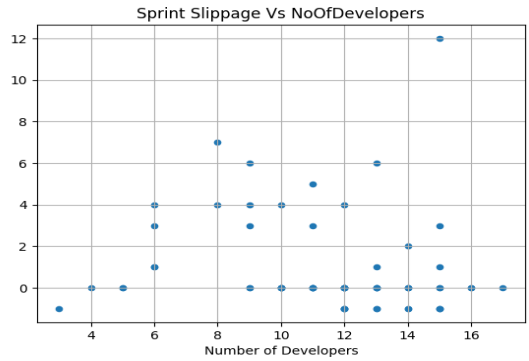

Figure 4 Sprint Slippage vs Number of Developers

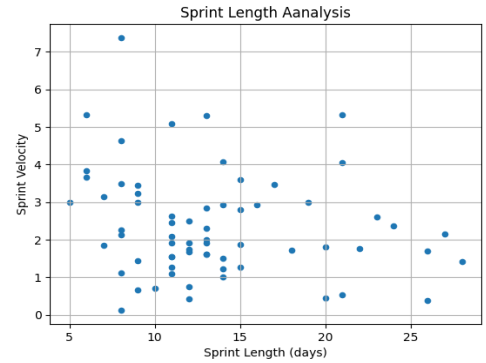

Figure 5 Sprint Length Analysis

iv. Sprint Length Analysis

The Sprint Length Analysis highlights the impact of sprint duration on sprint velocity. Fig 5 shows sprint velocity against sprint length to identify the optimal sprint duration for maximizing productivity. As shown in the graph generally, longer sprints are associated with lower sprint velocities, indicating that extended time frames may lead to decreased productivity or increased scope creep. This observation underscores the importance of establishing appropriate sprint lengths to maintain a balance between project predictability and delivery speed.

V. Issue Type Analysis

The focus of the Issue type analysis is to inspect the distribution of issue types (e.g., bugs, user stories, tasks) within each sprint and inspect the percentage of completed issues by sprint to examine how the type and the number of issues affects compilation percentage of issues in each sprint. Fig 6 shows the count of different issues for each sprint and Fig 7 shows compilation percentage of issues in each sprint, it compares the total number of committed issues (planned for the sprint) with the number of completed issues.

The Issue Type Analysis provides teams with actionable insights into how different types of issues impact sprint compilation. By understanding the distribution of issue types and their contribution to sprint completion, teams can optimize their workflows, prioritize tasks effectively, and drive continuous improvement in their development processes.

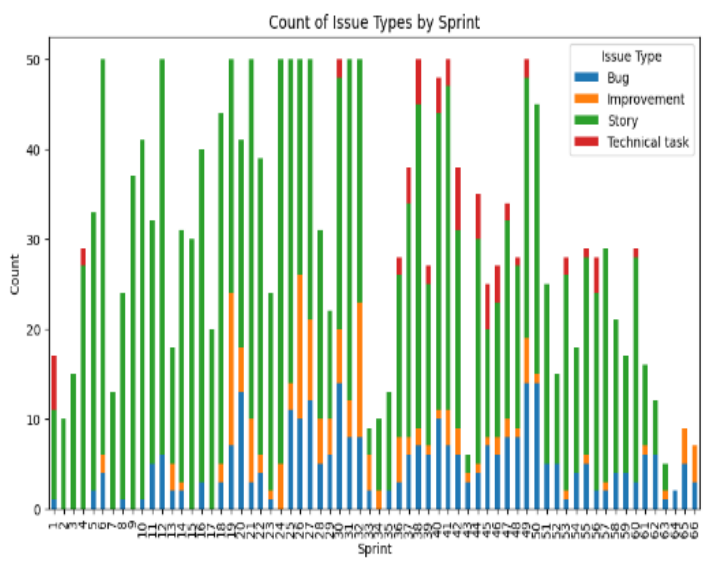

Figure 6 Count of Issue type by Sprint

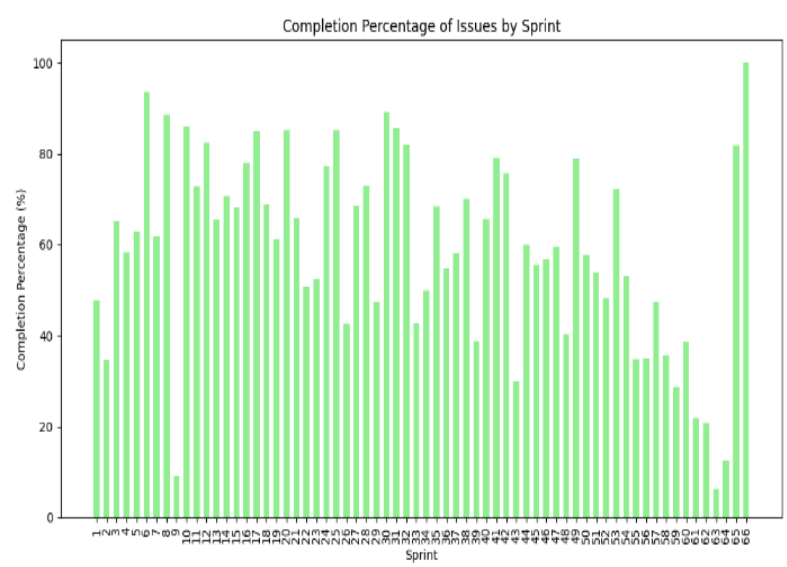

Figure 7 Completion of Percentage of Issues by Sprint

4.2.Log Data Analysis

A. Data Set

This dataset captures logs from a distributed system, providing a comprehensive view of system behavior and performance. The logs encompass a range of activities, including system events, errors, and performance metrics, offering valuable insights for understanding and optimizing distributed system architectures. The File Format is CSV. The columns are Timestamp, LogLevel, Service, Message, RequestID, User, ClientIP, TimeTaken.

B. Analysis

I. Log level by Service Analysis

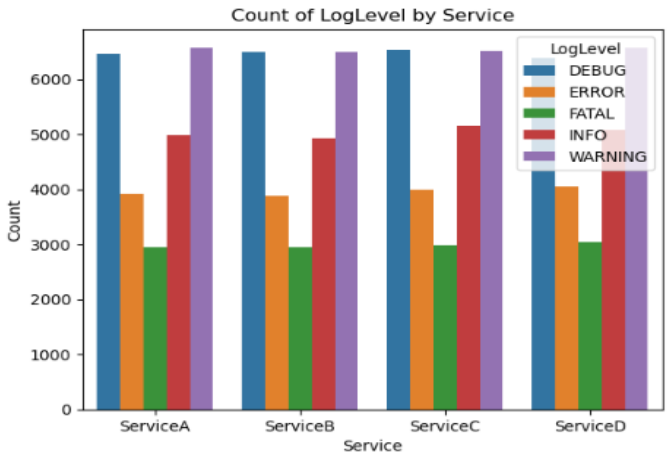

Figure 8 counts of each logleve for each service

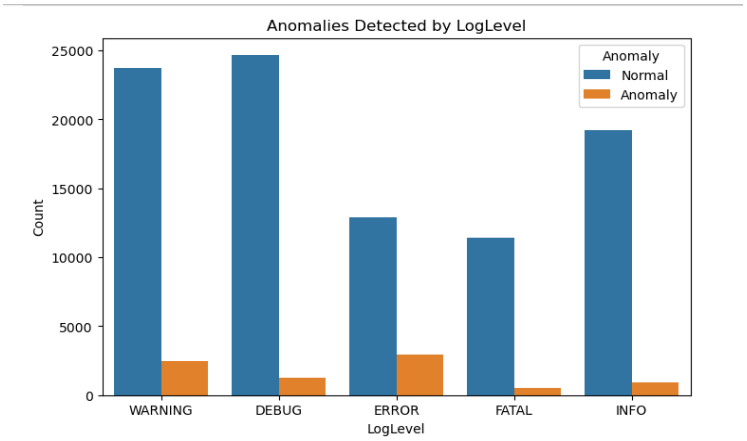

Figure 9 anomalies detected in each log level

Figure 8 provides valuable insights into the software development process using big data analytics.

a) Service-wise Log Level Distribution: The graph allows us to compare the distribution of log levels across different services. It provides an overview of the log levels that occur most frequently for each service, indicating areas where attention may be required.

b) Identifying Service-Specific Issues: By analyzing the log level distribution, we can identify services that consistently exhibit high log levels, indicating potential areas of concern. This information can be used to prioritize software development efforts and allocate resources to address the identified issues.

c) Log Level Imbalance: Some services may have a significantly higher count of log levels compared to others. This suggests that certain services may have more frequent issues or require additional monitoring and debugging.

d) Severity Assessment: The distribution of log levels across services helps assess the severity of issues encountered during software development. For example, if the majority of log levels are of high severity (e.g., errors or critical warnings), it indicates that certain services require immediate attention and debugging.

II. Anomalies by Log Levels analysis

The visualization in figure 9 represents the anomalies detected by log level in the log data. The x-axis shows different log levels, including INFO, DEBUG, WARNING, and ERROR, FATAL, while the y-axis represents the count of anomalies detected. The anomalies are categorized as 'Normal' and 'Anomaly,' where 'Normal' indicates regular system behavior and 'Anomaly' highlights potential issues or irregularities.

From the graph, we observe that anomalies are distributed across different log levels. The majority of anomalies are concentrated in the ERROR log level, indicating critical issues or errors in the system. This is followed by the WARNING log level, suggesting potential performance or resource warnings. The FATAL log level has relatively fewer anomalies, indicating fewer irregularities in normal system operations at this level.

The hue differentiation between 'Normal' and 'Anomaly' helps in visually identifying the distribution of anomalies within each log level. This visualization aids in understanding the severity and frequency of anomalies across the system's logging hierarchy.

III. Services' Anomalies Analysis

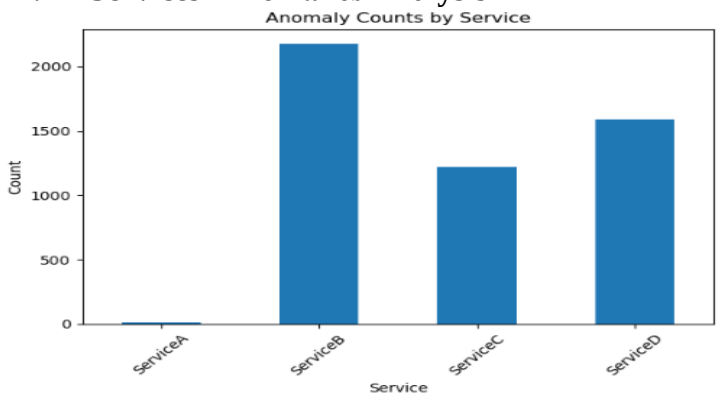

Figure 10 anomaly counts for different services

```
Service
ServiceA      14
ServiceB    2178
ServiceC    1218
ServiceD    1590
dtype: int64
```

```
        HourOfDay  Anomaly
81097           9       -1
81105           9       -1
81107           9       -1
81108           9       -1
81110           9       -1
...           ...      ...
99981           9       -1
99984           9       -1
99990           9       -1
99998           9       -1
99999           9       -1

[8100 rows x 11 columns]
```

Figure 11 lists entries that are anomalies in the log

Figure 10 shows anomaly counts for different services, serviceA has 14 anomalies, serviceB has 2178 anomalies, serviceC has 1218 anomalies, serviceD has 1590 anomalies.

The analysis reveals the number of anomalies detected in different services during the software development process. Anomalies can represent unexpected or irregular behavior in the software system. ServiceB has the highest number of anomalies (2178), followed by ServiceD (1590) and ServiceC (1218). ServiceA has the lowest number of anomalies (14). Higher anomaly counts may indicate potential issues or areas of **concern** in the corresponding services. The magnitude of the anomaly count can give insights into the relative complexity or stability of the services.

IV. User Behavior Analysis

Figure 12 shows the distribution of activities among different users. It helps visualize which users are more active or less active based on the frequency of their activities.

Figure 13 is the result of Suspicious Activity Detection. Our code identifies users with unusually high activity levels (activity count > 100). These users might require further investigation as their activity levels are significantly different from others, which could indicate potential suspicious activities. The result can be strengthened by using ClientIP Analysis. It provides insights into which IP addresses are more active, potentially indicating multiple users sharing the same IP or suspicious activities from specific IP ranges.

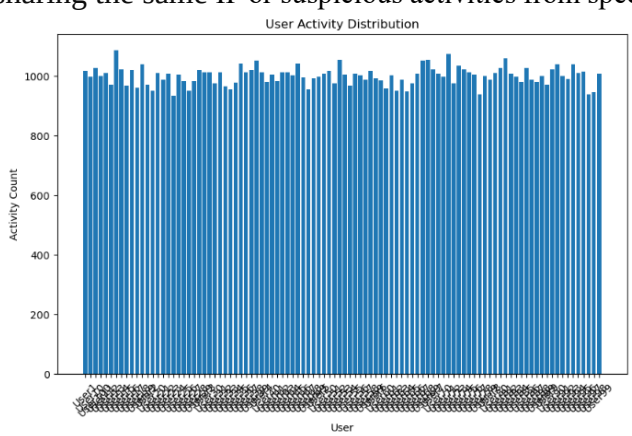

Figure 12 distribution of activities among different users

```
Suspicious Users:
       User  ActivityCount
0     User1           1018
1    User10            997
2   User100           1028
3    User11           1000
4    User12           1010
..      ...            ...
95   User95           1010
96   User96           1014
97   User97            939
98   User98            946
99   User99           1006

[100 rows x 2 columns]
```

Figure 13 counts of Suspicious user Activity

V. performance optimization using log data Analysis

```
Services with High Response Times:
Empty DataFrame
Columns: [mean, max, min]
Index: []

Services with Significant Variations in Response Times:
                mean  max  min
Service
ServiceA  55.197179  100   10
ServiceB  55.120969  100   10
ServiceC  55.025635  100   10
ServiceD  54.872191  100   10
```

The above output shows the following two points:

Services with High Response Times: No services were identified with an average response time above the specified threshold (threshold_mean). This suggests that, on average, the services in the dataset are meeting the performance expectations in terms of response times.

Services with Significant Variations in Response Times: Several services (ServiceA, ServiceB, ServiceC, ServiceD) exhibit significant variations between their maximum and minimum response times. The mean response times for these services range from approximately 55 to 55.2 units.The maximum response times for these services range from 10 to 100 units, indicating a wide range of response times observed within each service. The minimum response times for these services are consistently around 10 units.

Based on the two points raised, the majority of services in the dataset have response times that are within acceptable limits. Services ServiceA, ServiceB, ServiceC, and ServiceD show notable variations in their response times, with response times ranging from 10 to 100 units. Further investigation may be warranted for services with significant variations to understand the underlying factors contributing to these fluctuations and consider optimization strategies to improve consistency and efficiency in response times across these services.

VI. Predictive Maintenance

```
Accuracy: 1.00
Classification Report:
              precision    recall  f1-score   support

       100ms       1.00      1.00      1.00       210
        10ms       1.00      1.00      1.00       205
        11ms       1.00      1.00      1.00       190
        12ms       1.00      1.00      1.00       223
        13ms       1.00      1.00      1.00       231
    accuracy                           1.00     20000
   macro avg       1.00      1.00      1.00     20000
weighted avg       1.00      1.00      1.00     20000
```

```
Predicted Maintenance Needs (1) or Not (0):
[0 1 1 ... 0 0 0]
```

Predictive maintenance, typically refers to the predictions made by machine learning models regarding the maintenance needs or failure likelihood of a system or service. These predictions are based on historical data, operational parameters, and other relevant features. In our analysis we use Random Forest classifiers for identifying or predicting if maintenance is needed or not. The Random Forest classifier achieved an accuracy score of “1” on the test data, indicating high accuracy in predicting maintenance events. The classification report provides detailed metrics such as precision, recall, and F1-score for each class, which are crucial for evaluating the model's predictive performance. In our work Random Forest Classifier is trained using features such as log level, service, time taken, and message. The target variable (y) is likely a binary variable indicating whether a maintenance event occurred (e.g., 1 for maintenance needed, 0 for no maintenance needed). After training the model, it is used to predict maintenance events on unseen data (testing set). The predictions (y_pred) are essentially the model's estimates or forecasts about whether a certain system or service will require maintenance in the future, based on the provided features and historical patterns in the data. We create binary labels (maintenance_needed) where 1 indicates maintenance is needed and 0 indicates maintenance is not needed based on the threshold. Based on the model's predictions and feature importance analysis, it's recommended to focus on proactive maintenance strategies during specific hours of the day or when certain error log levels are prevalent.

5. Discussion and Conclusion

5.1.Discussion

Agile Scrum Velocity Dataset analysis profoundly impacts software development teams by identifying workflow inefficiencies and improvement opportunities. Trends like fluctuating sprint velocities and team composition differences highlight areas for enhancement. Insights from sprint slippage and issue type analysis inform proactive planning and task prioritization, which optimizes workflows.

The utilization of Agile Scrum Velocity Dataset also empowers teams to make informed decisions and enhance planning processes. Historical performance insights aid in setting realistic goals and resource allocation. Understanding team dynamics and anticipating potential delays enables proactive risk mitigation. In summary, Agile Scrum Velocity Dataset drives continuous improvement, streamlines workflows, and ensures software delivery that meets stakeholder expectations.

The visualization of log level counts and anomalies provides critical insights into the software development process and system health. By leveraging big data analytics, we can identify service-specific issues, prioritize development efforts, and allocate resources effectively. Monitoring anomalies, especially in ERROR and WARNING log levels, allows real-time troubleshooting and evaluation of development effectiveness. This approach improves overall efficiency, quality, system reliability, and performance, ensuring a seamless user experience.
User Behavior Patterns Analysis helps to understand activity distributions among users and IP addresses, aiding in identifying outliers or patterns that may require attention. Suspicious Activity Detection, based on predefined thresholds, flags users with unusual activity levels, serving as an initial step in detecting potentially suspicious activities for further investigation.

The analysis of service response times shows that while average response times meet expectations, significant variations exist in services like ServiceA, ServiceB, ServiceC, and ServiceD, possibly due to load variability, resource allocation, or service dependencies. To optimize performance, we recommend real-time monitoring, load balancing, bottleneck analysis, and regular maintenance, which will enhance service consistency, efficiency, and system reliability. Additionally, our predictive maintenance analysis using Random Forest classifiers accurately predicts maintenance needs by training on features such as log level, service, time taken, and message. This proactive approach helps prevent system failures, optimize resource utilization, and further improve system performance and reliability.

5.2.Conclusion

Our research demonstrates the profound impact of integrating big data analytics into software development. By analyzing Agile Scrum Velocity datasets, we identified critical patterns and trends, such as fluctuations in sprint velocity and the effects of team composition on productivity. These insights enable teams to optimize workflows, prioritize tasks effectively, and set realistic project goals. Additionally, our examination of system log data provided valuable information for detecting anomalies, understanding user behavior, and enhancing service response times. The implementation of predictive maintenance strategies using machine learning models further underscores the potential of big data analytics to improve system reliability and performance. Overall, this study highlights the significant benefits of leveraging data-driven approaches to drive continuous improvement and innovation in software development.

6. Limitations and Future Works

6.1 Limitations

While our research aimed to utilize big data analytics to enhance software development practices, some limitations affected the scope and depth of our study:

1. **Data Scale**: Due to time and resource constraints, the datasets used in our research do not represent true big data. This limitation restricted the extent to which we could explore and leverage advanced big data analytics techniques.
2. **Single Project Data**: Our analysis of Agile Scrum Velocity and Log files was limited to data from a single project. This narrow focus may limit the generalizability of our findings. Incorporating datasets from multiple projects and diverse sources would provide a more comprehensive understanding of Scrum velocity dynamics across different contexts.
3. **Scope of Analysis**: The analysis performed was constrained by the available data, tools and methodologies. Broader and more varied analytical techniques could offer deeper insights and more robust conclusions.

6.2. Future Work

To address the above limitations and build upon our findings, we will consider the following directions:

1. **Utilizing True Big Data**: Expanding the scope to include larger datasets that embody the characteristics of big data (volume, velocity, variety, veracity, and value) will allow for more thorough and nuanced analysis.

Leveraging distributed computing platforms such as Hadoop or Spark could facilitate the processing and analysis of these extensive datasets.

2. **Diverse Data Sources**: Incorporating Agile Scrum Velocity datasets from multiple projects and organizations. This would enable a more generalized understanding of the patterns and factors influencing Scrum velocity, enhancing the applicability and reliability of the research outcomes.
3. **Advanced Analytical Techniques**: Employing a broader range of analytical methods, including machine learning algorithms, and advanced statistical techniques, could provide deeper insights into both Scrum velocity and log data. These approaches could uncover more subtle patterns and relationships that were not identified in the current study.

References

1. Srivastava, A., Bhardwaj, S., & Saraswat, S. (2017, February). SCRUM Model for Agile Methodology. In Proceedings of the International Conference on Computing, Communication and Automation (ICCCA 2017)

2.Pomar, F. A., Calvo-Manzano, J. A., Caballero, E., Arcilla-Cobián, M. (2014). Understanding sprint velocity fluctuations for improved project plans with Scrum: a case study. Information and Software Technology, 56(11), 1486-1498.

3. Randula Koralage,. Agile Scrum Sprint Velocity Dataset. Retrieved from https://github.com/RandulaKoralage/AgileScrumSprintVelocityDataSet/tree/master/Agile%20Scrum%20Dataset/Finalized%20 Datasets%20for%20 Spring%20XD%20Project%20by%20Randula

4.https://www.kaggle.com/datasets/shubhampatil1999/synthetic-log-data-of-distributed-system

5. Boyuan C.(2019). Improving the software logging practices in DevOps.

6. Shenghui Gu , Guoping Rong , He Zhang, and Haifeng Shen(2023). Logging Practices in Software Engineering: A Systematic Mapping Study

7. Batarseh, Feras & Gonzalez, Avelino. (2015). Predicting failures in agile software development through data analytics. Software Quality Journal. 26. 10.1007/s11219-015-9285-3.

8.Batarseh, F., & Gonzalez, A. (2015). Predicting failures in agile software development through data analytics.*Software Quality Journal*, *26*(1), 1-18. [DOI: 10.1007/s11219-015-9285-3]

7. Appendix

1.Sample Agile Scrum Velocity Dataset

sprintId	sprintName	sprintState	sprintStartDate	sprintEndDate	sprintCompleteDate	totalNumberOfIssues	completedIssuesCount	issuesNotCompletedInCurrentSprint	puntedIssues	issuesCompletedInAnotherSprint	issueKeysAddedDuringSprint	completedIs
1	Sprint 1	CLOSED	15-Apr-13	29-Apr-13	6-May-13	23	11	0	6	0	15	12
3	Sprint 2	CLOSED	6-May-13	19-May-13	14-May-13	26	9	0	8	1	7	10
4	Sprint 3	CLOSED	15-May-13	20-May-13	20-May-13	23	15	0	4	0	1	70
5	Sprint 4	CLOSED	20-May-13	24-May-13	28-May-13	48	28	0	10	0	17	63
6	Sprint 5	CLOSED	28-May-13	3-Jun-13	3-Jun-13	35	22	3	1	8	7	65
8	Sprint 6	CLOSED	3-Jun-13	12-Jun-13	17-Jun-13	61	57	0	1	2	33	150
9	Sprint 7	CLOSED	18-Jun-13	21-Jun-13	25-Jun-13	21	13	0	4	0	7	51
10	Sprint 8	CLOSED	25-Jun-13	28-Jun-13	1-Jul-13	26	23	0	1	1	6	116
11	Sprint 9	CLOSED	1-Jul-13	5-Jul-13	8-Jul-13	241	22	12	102	3	108	49
13	Sprint 10	CLOSED	8-Jul-13	12-Jul-13	16-Jul-13	43	37	3	1	1	9	75
15	Sprint 11	CLOSED	16-Jul-13	19-Jul-13	22-Jul-13	44	32	0	6	0	8	68
16	Sprint 12	CLOSED	22-Jul-13	31-Jul-13	12-Aug-13	136	112	0	12	0	90	271
18	Sprint 13	CLOSED	12-Aug-13	20-Aug-13	20-Aug-13	26	17	0	4	1	13	52
19	Sprint 14	CLOSED	20-Aug-13	29-Aug-13	29-Aug-13	41	29	0	5	2	12	108
20	Sprint 15	CLOSED	29-Aug-13	5-Sep-13	11-Sep-13	44	30	0	7	0	22	113
21	Sprint 16	CLOSED	18-Sep-13	10-Oct-13	10-Oct-13	50	39	0	5	1	26	142
24	Sprint 17	CLOSED	11-Oct-13	18-Oct-13	22-Oct-13	20	17	3	0	0	4	124
25	Sprint 18	CLOSED	23-Oct-13	12-Nov-13	18-Nov-13	64	44	0	10	0	30	213

The sprint data set 67.CSV includes attributes such as sprintId and sprintName, as well as time-related data such as sprintStartDate and sprintEndDate. Additionally, the dataset includes metrics related to issue management, such astotalNumberOfIssues, completedIssuesCount, and issuesNotCompletedInCurrentSprint, which offer insights into the progress and completion status of tasks within each sprint. Other key metrics, such as punted issues and puntedIssuesEstimateSum, provide further context on sprint performance and potential challenges faced by the development team. Overall, the sprint dataset serves as a valuable resource for analyzing sprint dynamics, identifying trends, and evaluating the effectiveness of the software development process within an agile framework.

The other two datasets named issues 1992.csv contain detailed information about individual issues within a software development project. Each row represents a specific issue, and the dataset includes various attributes describing the characteristics and status of each issue. And Issues Summary 2861.csv data set provides summarized information about issues within each sprint of the project. Each row represents a summarized view of issues within a specific sprint. These datasets provide valuable insights into the issues encountered during the development process, their status, priority, and assignment, facilitating effective issue tracking and management within the project.

2. Sample Dataset for the Log data of distributed system

jupyter logdata.csv Last Checkpoint: 5 months ago

File Edit View Settings Help

Delimiter: ,

		Timestamp	LogLevel	Service	Message	RequestID	User	ClientIP	TimeTaken
1	0	023-11-20T08:40:50.664842	WARNING	ServiceA	Performance Warnings	6743	User96	192.168.1.102	28ms
2	1	023-11-20T08:40:50.672154	DEBUG	ServiceA	File I/O	8684	User17	192.168.1.219	55ms
3	2	023-11-20T08:40:50.680263	WARNING	ServiceA	Performance Warnings	6265	User13	192.168.1.173	44ms
4	3	023-11-20T08:40:50.688973	ERROR	ServiceA	Critical Errors	5821	User1	192.168.1.185	72ms
5	4	023-11-20T08:40:50.697002	ERROR	ServiceB	Critical Errors	7272	User64	192.168.1.194	56ms
6	5	023-11-20T08:40:50.705617	FATAL	ServiceA	Crashes	7983	User48	192.168.1.129	22ms
7	6	023-11-20T08:40:50.714641	WARNING	ServiceD	Performance Warnings	3378	User40	192.168.1.56	12ms
8	7	023-11-20T08:40:50.729253	WARNING	ServiceB	Performance Warnings	2061	User15	192.168.1.58	11ms
9	8	023-11-20T08:40:50.742986	ERROR	ServiceD	Critical Errors	9966	User39	192.168.1.219	52ms
10	9	023-11-20T08:40:50.759240	ERROR	ServiceB	Database Errors	3059	User57	192.168.1.97	79ms
11	10	023-11-20T08:40:50.783024	WARNING	ServiceD	Resource Warnings	4230	User55	192.168.1.94	64ms
12	11	023-11-20T08:40:50.798636	INFO	ServiceC	Startup Messages	6556	User68	192.168.1.23	86ms
13	12	023-11-20T08:40:50.814813	DEBUG	ServiceD	Trace Information	7745	User89	192.168.1.20	24ms
14	13	023-11-20T08:40:50.830576	DEBUG	ServiceD	File I/O	7566	User95	192.168.1.170	64ms
15	14	023-11-20T08:40:50.846785	FATAL	ServiceC	Data Corruption	7077	User20	192.168.1.110	83ms
16	15	023-11-20T08:40:50.857131	WARNING	ServiceC	Resource Warnings	4703	User84	192.168.1.117	61ms
17	16	023-11-20T08:40:50.863332	FATAL	ServiceC	Data Corruption	2389	User58	192.168.1.102	83ms
18	17	023-11-20T08:40:50.878447	INFO	ServiceA	Status Updates	7751	User48	192.168.1.10	16ms

Column Description:

Timestamp : Records the date and time of each logged event .
LogLevel : Indicates the severity or importance(INFO, WARNING, ERROR, or FATAL)
Service : Specifies the name or identifier of the service associated with each log entry.
Message : Contains descriptive information or details related to the logged event.
RequestID : Uniquely identifies each request.
User : Represents the user associated with the logged event.
ClientIP : Uniquely identifies the client or application associated with the logged event.
TimeTaken : Records the duration, in milliseconds or another specified unit.